

Autonomous OS Debugging Agent

200 Questions & Answers — Easy Language Guide

Team	KernelHealers
Members	Ansh (Leader) · Swati · Aman · Shivang
Hackathon	Build With Bharat 2.0
Theme	Blockchain & Web3 — Agentic Solutions (x402)
Blockchain	Algorand TestNet lora.algokit.io/testnet
GitHub	github.com/Ansh00031/SGU-AI-Thon
Total Questions	200 across 10 categories with full easy-language answers

■ Contents

- CATEGORY 1: Basic Project Understanding (20 questions)
- CATEGORY 2: AI Engine & Reasoning Loop (30 questions)
- ■ CATEGORY 3: Security & Safety (30 questions)
- CATEGORY 4: Sessions, Snapshots & Rollback (20 questions)
- CATEGORY 5: Auto-Start & Reboot Management (20 questions)
- CATEGORY 6: WinRE Cloud Recovery (20 questions)
- CATEGORY 7: Blockchain & Algorand (30 questions)
- CATEGORY 8: Enterprise & Real-World Impact (15 questions)
- CATEGORY 9: Future Scope & Roadmap (10 questions)
- CATEGORY 10: Judge Q&A; — Top 5 Critical Questions (5 questions)

■ CATEGORY 1: Basic Project Understanding

Q1. What is the Autonomous OS Debugging Agent?

Ans: It is an AI-powered tool that runs in your terminal. When your Windows PC gets an error, this agent automatically figures out what went wrong, suggests a fix, and applies it safely — just like having a computer expert sitting inside your laptop.

Q2. What problem does this project solve?

Ans: When a PC gets OS errors like '0x80070005', most people spend 30–45 minutes Googling random fixes and running risky commands. This tool solves the problem in under 15 seconds by doing all that work automatically.

Q3. Why can't users just use ChatGPT to fix OS errors?

Ans: ChatGPT guesses commands without actually checking what is wrong on YOUR machine. It can suggest a fix that breaks your PC further. Our agent first reads real live data from your system, then suggests a fix — so there are no guesses.

Q4. What does the term 'autonomous' mean in this context?

Ans: Autonomous means the agent works on its own. It collects data, runs safe checks, generates a fix, and verifies it worked — all without you having to tell it each step. You only need to approve the fix before it runs.

Q5. What is an OS error code like 0x80070005?

Ans: It is a code that Windows shows when something goes wrong. The numbers and letters represent a specific type of error. '0x80070005' specifically means 'Access Denied' — some permission or security setting is blocking an operation.

Q6. What does '0x80070005: Access Denied' mean on Windows?

Ans: It means Windows tried to do something (like update itself or start a service) but was blocked because the required permissions were missing or broken. Usually caused by damaged NTFS ACL (Access Control List) settings.

Q7. Who is the target user for this tool?

Ans: Anyone who uses Windows — from home users whose PC crashes, to IT teams in companies managing hundreds of computers. No technical knowledge is required to use it.

Q8. Is this project open-source? Where is the code?

Ans: Yes, it is 100% open-source. The full code is available on GitHub at: github.com/Ansh00031/SGU-AI-Thon

Q9. What operating systems does it support?

Ans: Currently it fully supports Windows 10 and Windows 11. Linux and macOS support is planned for future versions.

Q10. What programming language is the project built in?

Ans: The entire project is built in Python 3.10+. The CLI uses the 'Typer' library and the terminal display uses 'Rich' for colored tables and panels.

Q11. What is the 'AI mechanic inside your terminal' analogy?

Ans: Think of it like calling a car mechanic. Instead of you guessing what is wrong with your car, the mechanic plugs in a diagnostic tool, reads the error codes, and fixes the specific problem. Our agent does the same for your PC.

Q12. How is this different from a standard antivirus tool?

Ans: Antivirus tools scan for viruses and malware. Our tool fixes OS-level errors like broken permissions, failed Windows Updates, stopped services, and registry issues. These are system health problems, not security threats.

Q13. How long does a traditional IT support ticket take vs. your tool?

Ans: A traditional IT ticket takes 30–45 minutes to hours. Our agent diagnoses and fixes the same error in under 15 seconds with full AI reasoning and safe execution.

Q14. What is the main CLI command to start diagnosis?

Ans: You open a terminal and type: `python agent.py diagnose 0x80070005` (replace 0x80070005 with your actual error code). That is all you need to type.

Q15. Does the user need to know PowerShell to use this tool?

Ans: No. The agent generates and runs all PowerShell commands internally. You just approve or reject the fix by pressing Y or N. No scripting knowledge needed.

Q16. What is the minimum Python version required?

Ans: Python 3.10 or higher is required. The project uses modern Python features like union types and improved pattern matching.

Q17. Does the tool require internet to work?

Ans: For the AI reasoning (GPT-4o or similar), yes — internet is needed. However, it can also run in offline mode using a local AI model like Ollama (Llama 3.1) without any internet.

Q18. Can it work fully offline?

Ans: Yes. By configuring a local AI model via Ollama, the agent can diagnose and fix errors completely offline. The only internet-requiring feature is the Algorand blockchain anchor.

Q19. What is the GitHub repository link for this project?

Ans: github.com/Ansh00031/SGU-AI-Thon — contains full source code, documentation, PDF, presentation, and the 1-line WinRE rescue command.

Q20. What was the inspiration behind building this?

Ans: The team experienced Windows Update failures and OS errors that wasted hours of time. The idea was to build an autonomous AI agent that solves these problems the same way an expert IT engineer would — but in seconds.

■ CATEGORY 2: AI Engine & Reasoning Loop

Q21. What AI model powers the diagnostic reasoning?

Ans: By default it uses OpenAI GPT-4o. But it can be configured to use any OpenAI-compatible model including Ollama (Llama 3.1, Mistral, DeepSeek) for fully local/offline operation.

Q22. What is 'hallucination' in AI and why is it dangerous for OS repair?

Ans: Hallucination is when AI confidently gives wrong answers. In OS repair, this is very dangerous — if the AI suggests a wrong PowerShell command, it could delete system files or break Windows completely.

Q23. How does the agent avoid hallucinations?

Ans: Before suggesting any fix, the agent first reads REAL live data from your system (Event logs, service status, permissions). The fix is grounded in actual evidence — not guesswork. This is called Active Telemetry Grounding.

Q24. What is the Active Telemetry Reasoning Loop?

Ans: It is the 5-step process where the agent: (1) reads system data, (2) forms a hypothesis, (3) runs safe diagnostic probes, (4) confirms the root cause from real evidence, then (5) generates a targeted fix.

Q25. What are the 5 stages of the AI reasoning pipeline?

Ans: Stage 1: Hypothesis Formulation. Stage 2: Diagnostic Command Generation. Stage 3: Root Cause Confirmation. Stage 4: Remediation Script Synthesis. Stage 5: Verification Evaluation.

Q26. What is 'hypothesis formulation' in Stage 1?

Ans: The AI looks at your error code and system data and makes an educated guess about the possible causes. For example, '0x80070005 could be caused by broken NTFS permissions on the Windows Update folder'.

Q27. What are read-only diagnostic probes?

Ans: These are safe commands that only READ information from your system — they never change anything. For example, 'icacls C:\Windows\System32' only checks permissions but never modifies them.

Q28. What is icacls and what does it check?

Ans: icacls is a Windows built-in command that shows file and folder permissions (who can read, write, or execute a file). The agent uses it to check if important Windows folders have correct permissions.

Q29. What is Get-Service used for in diagnostics?

Ans: Get-Service is a PowerShell command that lists the status of Windows services (like Windows Update, BITS, Cryptographic Services). The agent uses it to see if critical services are running or stopped.

Q30. What is Get-WinEvent and which logs does it read?

Ans: Get-WinEvent reads Windows Event Viewer logs. The agent reads from System, Application, and WindowsUpdateClient channels to see exactly what errors occurred and when.

Q31. How does the AI confirm the root cause in Stage 3?

Ans: After the safe diagnostic commands run, their output is fed back to the AI. The AI reads this evidence and narrows down the exact cause. For example: 'The WindowsUpdate service is stopped AND the BITS service has Access Denied errors — root cause confirmed.'

Q32. What information is fed back to the AI after probes run?

Ans: The command output (text results), exit codes (success/failure numbers), any error messages, and the purpose of each command are all sent back to the AI for analysis.

Q33. What is a 'remediation script' in Stage 4?

Ans: It is a PowerShell script that fixes the confirmed root cause. For example, if permissions are broken, the script runs 'icacls' to reset them. The script is targeted — it only fixes what is actually broken.

Q34. What language are remediation scripts written in?

Ans: PowerShell (.ps1 scripts) for Windows. PowerShell is Windows' built-in scripting language and has full access to system settings, services, and registry without needing extra software.

Q35. How does the AI structure its JSON output for fix proposals?

Ans: The AI returns a structured JSON object containing: title (fix name), script_content (the actual PowerShell code), verification_command (to check if fix worked), requires_reboot (true/false), and risk_level.

Q36. What is the verification command in Stage 5?

Ans: After the fix runs, the agent automatically runs a verification command (like 'Get-Service wuauserv | Select Status') to confirm the problem is actually solved. The AI then evaluates if the result shows success.

Q37. Can the AI work without an OpenAI API key?

Ans: Yes. You can configure it to use a local AI model via Ollama. Set OPENAI_BASE_URL=http://localhost:11434/v1 in the .env file and it will use your local Llama or Mistral model completely offline.

Q38. What happens in offline/heuristic mode without an API key?

Ans: The agent falls back to built-in expert heuristic rules that cover the most common Windows error patterns. It is less intelligent than AI mode but still functional for standard errors.

Q39. Can you use a local LLM like Ollama or Llama 3.1?

Ans: Yes. Install Ollama, pull a model (ollama pull llama3.1), then set OPENAI_BASE_URL=http://localhost:11434/v1 in the .env file. The agent will use your local model for 100% private offline operation.

Q40. How do you configure Ollama as the AI backend?

Ans: In the .env file: OPENAI_BASE_URL=http://localhost:11434/v1 and LLM_MODEL=llama3.1. No API key needed for local Ollama models.

Q41. What is the OPENAI_BASE_URL setting used for?

Ans: It tells the agent where to send AI requests. By default it goes to OpenAI's servers. Change it to point to your local Ollama, Azure OpenAI, or any other OpenAI-compatible endpoint.

Q42. What is the LLM_MODEL environment variable?

Ans: It specifies which AI model to use. Default is 'gpt-4o'. For local models you'd set it to 'llama3.1' or 'mistral' depending on what you have installed.

Q43. How does the agent handle LLM rate limits or timeouts?

Ans: If the AI service is slow or rate-limited, the agent will display an error and fall back to the heuristic offline mode so the diagnosis can still proceed.

Q44. What is a JSON Schema prompt and why use strict JSON?

Ans: The agent tells the AI to return answers in a strict JSON format (like a form). This prevents the AI from returning random text and ensures the fix proposal is always machine-readable and safe to process.

Q45. What happens if the AI returns malformed JSON?

Ans: The agent has a fallback JSON extractor that searches for JSON-like structures in the AI response. If it still cannot parse it, the agent displays a warning and asks the user to retry.

Q46. How does the agent ensure the AI stays on topic?

Ans: The system prompt explicitly tells the AI its role (OS diagnostic expert), what tools it has (safe probes only), what is forbidden (destructive commands), and what format to respond in. This constrains the AI to safe, relevant responses.

Q47. Can the AI diagnose Linux errors too?

Ans: Not yet. Currently it is Windows-only. Linux support (systemd, journalctl, apt errors) is planned for the next version.

Q48. What Windows Event Viewer channels does the agent read?

Ans: System (hardware and OS errors), Application (software errors), and Microsoft-Windows-WindowsUpdateClient/Operational (Windows Update specific errors).

Q49. How many event log entries does it collect by default?

Ans: 50 entries by default. You can change this with the --max-events flag (e.g., python agent.py diagnose 0x80070005 --max-events 100).

Q50. What is the --max-events flag used for?

Ans: It controls how many recent error log entries the agent reads from Windows Event Viewer. More entries = more context for AI = better diagnosis, but also slightly slower.

CATEGORY 3: Security & Safety

Q51. What is the Security Guardrail / Command Blacklist?

Ans: It is a built-in filter that automatically blocks any dangerous command before it can run. The agent checks every single command against a list of forbidden patterns. If a command matches, it is blocked immediately.

Q52. Which commands are blocked by the blacklist?

Ans: Commands like: del, rmdir, format, Remove-Item, reg delete, net user /delete, rd /s, cipher /w, and any command that could delete files, wipe drives, remove accounts, or damage the system.

Q53. How is the blacklist implemented technically?

Ans: Using Python regular expressions (regex). Each command is tested against patterns like `r'\bde\b'` or `r'\bformat\b'`. If any pattern matches, the command is rejected and logged.

Q54. Can the blacklist be bypassed?

Ans: During the safe diagnostic probe phase, no — the blacklist always runs. During the remediation phase, the AI is instructed to never include blacklisted commands, and even if it did, the user sees the script before execution and can reject it.

Q55. What is the Human-in-the-Loop (HITL) gate?

Ans: Before any fix runs, the agent shows you the complete PowerShell script with syntax highlighting and asks 'Do you want to execute this fix? [Y/n]'. Nothing happens unless you type Y and press Enter.

Q56. What exactly is shown to the user before executing a fix?

Ans: The full PowerShell script is displayed with colored syntax highlighting (Monokai theme), along with the fix title, risk level, what it will change, and what verification command will run afterward.

Q57. What happens if the user selects 'N' at the approval gate?

Ans: The agent exits cleanly. No changes are made to the system. The generated script is shown on screen so the user can copy it manually if they want to review or run it themselves.

Q58. What is a 'read-only diagnostic probe' vs. a 'write command'?

Ans: A read-only probe only reads information (like checking file permissions or service status). A write command changes something (like modifying a registry key). All diagnostic probes are strictly read-only. Only user-approved fix scripts can write.

Q59. How are diagnostic commands sandboxed?

Ans: Each command runs as a separate child process with a timeout limit. The output is captured and returned to the agent. The command cannot access the agent's memory or files.

Q60. What is the temp file sandbox execution model?

Ans: The fix script is saved to a temporary file (in %TEMP%), executed, and then immediately deleted after execution completes — win or lose. This prevents scripts from lingering on the system.

Q61. Are fix scripts auto-deleted after execution?

Ans: Yes. Using Python's NamedTemporaryFile with delete=True, the script file is guaranteed to be deleted even if the execution crashes or the user presses Ctrl+C.

Q62. What is a pre-fix snapshot?

Ans: Before any fix runs, the agent saves a copy of the current system state — including the fix script, a rollback script, session metadata, and the verification command — into a .backups folder.

Q63. Where are snapshots stored on disk?

Ans: In the .backups/ folder inside your project directory. Each session gets its own subfolder named by session ID, e.g.: .backups/session_20260818_224722_80070005/

Q64. What does a snapshot contain?

Ans: Four files: metadata.json (session info), fix.ps1 (the applied fix script), rollback.ps1 (the reversal script), and later blockchain_receipt.json (if anchored to Algorand).

Q65. What is a rollback script?

Ans: It is a PowerShell script that does the OPPOSITE of the fix. If the fix added a registry key, the rollback deletes it. If the fix reset permissions, the rollback restores the original values.

Q66. How is the rollback script generated?

Ans: The AI generates it at the same time as the fix script. It is specifically designed to undo every change the fix makes, in reverse order.

Q67. How long does a rollback take to execute?

Ans: Usually under 1 second for simple fixes. More complex fixes (like DISM repairs) might take a few seconds. The rollback is designed to be fast.

Q68. What is 'python agent.py rollback' and when do you use it?

Ans: Run this command if a fix made your PC worse or you changed your mind. It automatically finds the most recent session and runs its rollback script to undo all changes.

Q69. Can you rollback multiple sessions?

Ans: Yes. Use: python agent.py rollback session_ID to specify which session to roll back. Run python agent.py history to see all session IDs.

Q70. What happens if the rollback script itself fails?

Ans: The agent shows the error, displays the rollback script on screen, and suggests manual steps. The original fix script is also available in .backups/ so you can analyze what was changed.

Q71. What is Administrator privilege and why is it required?

Ans: Administrator (admin) privilege means the program has permission to change system settings. Without it, the agent cannot read certain protected Event Viewer logs or apply fixes to system folders.

Q72. What is --skip-admin-check flag and when should it be used?

Ans: This flag lets you run the agent WITHOUT admin rights — useful for testing or demos. In this mode, some system log access may be limited. Use: python agent.py diagnose 0x80070005 --skip-admin-check

Q73. What is dry-run mode?

Ans: Running with --skip-admin-check activates a limited dry-run mode where the agent reads what it can and generates fix proposals but may not be able to execute them on protected system areas.

Q74. Can a standard (non-admin) user run the agent?

Ans: Yes, with --skip-admin-check. But for full functionality — reading all Event Viewer logs and applying fixes — running as Administrator is recommended.

Q75. What are the most common dangerous commands blocked?

Ans: del, rmdir, format, Remove-Item -Recurse, reg delete, net user /delete, cipher /w (secure wipe), rd /s /q, and any command containing 'wipe', 'erase', or 'destroy'.

Q76. Does the agent access the internet during diagnosis?

Ans: Only to call the AI API (OpenAI/Ollama). All system diagnostic commands run locally. No system data is sent to the internet except what is sent to the AI for analysis.

Q77. Can the agent modify the Windows Registry?

Ans: Yes, but only through user-approved fix scripts. Diagnostic probes use Get-ItemProperty (read-only). Fix scripts may use Set-ItemProperty or reg add — but only after you approve them.

Q78. What is the difference between diagnosis phase and remediation phase?

Ans: Diagnosis = only reading/observing (100% safe, no changes). Remediation = applying the fix (requires admin, requires Y/N approval). These two phases are strictly separated.

Q79. How does the agent handle timeouts on diagnostic commands?

Ans: Each command has a 30-second timeout. If it doesn't finish in time, the agent marks it as timed out, skips it, and continues with other diagnostic commands.

Q80. What is the execute_diagnostic_command function?

Ans: It is the core function in core/executor.py that safely runs any diagnostic command: validates it against the blacklist, executes it with a timeout, captures output, and returns structured results.

CATEGORY 4: Sessions, Snapshots & Rollback

Q81. What is a session ID and how is it generated?

Ans: A session ID is a unique name for each diagnosis run. It is generated from the current date, time, and error code. Example: session_20260818_224722_80070005

Q82. Where are all sessions stored?

Ans: In the .backups/ folder inside your project directory. Each session has its own subfolder.

Q83. What is the .backups/ directory structure?

Ans: .backups/ contains one folder per session. Inside each session folder: metadata.json, fix.ps1, rollback.ps1, and optionally blockchain_receipt.json.

Q84. What files are inside a session's backup folder?

Ans: metadata.json (who, what, when), fix.ps1 (the PowerShell fix that was applied), rollback.ps1 (the undo script), blockchain_receipt.json (Algorand TX proof if anchored).

Q85. What is metadata.json in a session?

Ans: It stores all information about the session: error code, session ID, timestamp, fix title, status (completed/failed), verification command, and whether a rollback was executed.

Q86. What is fix.ps1 in a session?

Ans: It is the exact PowerShell script that was applied to fix the error. It is saved permanently so you can review exactly what changed on your system.

Q87. What is rollback.ps1 in a session?

Ans: It is the undo script. Running it reverses everything that fix.ps1 did. It is saved immediately when the fix is created, before any changes happen.

Q88. What is the 'history' command used for?

Ans: python agent.py history shows a table of all past diagnosis sessions — their ID, error code, fix title, status, and timestamp.

Q89. How do you view all past sessions?

Ans: Run: python agent.py history. You will see a formatted table with all sessions sorted by newest first.

Q90. What session statuses exist?

Ans: COMPLETED (fix applied successfully), FAILED (fix did not work), ROLLED_BACK (fix was reversed), VERIFIED_POST_REBOOT (verified after restart), PARTIAL (partially fixed), AWAITING_REBOOT (waiting for restart).

Q91. What is VERIFIED_POST_REBOOT status?

Ans: After a fix requires a restart, the agent automatically runs a verification check on the next boot. If it passes, the session status is updated to VERIFIED_POST_REBOOT.

Q92. What is blockchain_receipt.json inside a session?

Ans: If you used the --anchor flag or python agent.py blockchain anchor, this file stores the Algorand transaction ID, Lora Explorer URL, SHA-256 hash, and confirmation round.

Q93. Can you delete a session's backup manually?

Ans: Technically yes, but it is not recommended. Deleting it means you lose the rollback script and cannot undo that fix if something goes wrong later.

Q94. What happens to the session if rollback is executed?

Ans: The session status is updated to ROLLED_BACK and a rollback_executed: true flag is set in metadata.json. The session record is kept for audit purposes.

Q95. How many sessions are stored by default?

Ans: All sessions are stored indefinitely. There is no automatic cleanup or limit. You can manually delete old session folders from .backups/ if you want to free up space.

Q96. Is there a cleanup or expiry for old sessions?

Ans: Not currently. A cleanup command (python agent.py cleanup --older-than 30d) is planned for a future version.

Q97. What is the update_session_status function?

Ans: It is the function in core/snapshot.py that updates the status field in a session's metadata.json file. Called after fix execution and after rollback.

Q98. What is the get_session function?

Ans: It reads a session's metadata.json by session ID and returns the session data as a Python dictionary for the agent to work with.

Q99. What is the list_sessions function?

Ans: It scans the .backups/ directory, reads all session metadata files, and returns them as a sorted list (newest first) for display in the history command.

Q100. How does the rollback command find the most recent session?

Ans: It calls list_sessions() which returns sessions sorted by timestamp. The first item in the list is the most recent session. If no session ID is specified, it uses this one.

CATEGORY 5: Auto-Start & Reboot Management

Q101. What is the enable-autostart command?

Ans: python agent.py enable-autostart adds a batch file to the Windows Startup folder so the agent automatically runs startup-monitor every time the PC starts or restarts.

Q102. Where is the startup batch file created on Windows?

Ans: At: %APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\OS_Debug_Agent_Startup.bat — Windows automatically runs all files in this folder on boot.

Q103. What does the startup batch file do on boot?

Ans: It runs 'python agent.py startup-monitor' which checks for active system problems and shows a health report in a Command Prompt window that stays open for the user to see.

Q104. What is the disable-autostart command?

Ans: python agent.py disable-autostart removes the startup batch file, so the agent no longer runs automatically on boot.

Q105. What is the startup-monitor command?

Ans: python agent.py startup-monitor shows a dashboard with two sections: Active Problems (current errors not yet fixed) and Solved Problems (successfully fixed errors). Also shows live service health.

Q106. What is the difference between Active Problems and Solved Problems?

Ans: Active Problems = sessions with FAILED or PARTIAL status (still need attention). Solved Problems = sessions with COMPLETED or VERIFIED status (successfully fixed). This gives you a quick health overview on every boot.

Q107. What services does startup-monitor check live?

Ans: wuauserv (Windows Update), bits (Background Intelligent Transfer Service), and cryptsvc (Cryptographic Services). These are the three most critical services for Windows health.

Q108. What is the startup-log command?

Ans: python agent.py startup-log reads and displays the .backups/startup_log.txt file, which records every time the startup health check ran — with timestamps and results.

Q109. Where is the startup execution log stored?

Ans: At: .backups/startup_log.txt inside the project directory. Each startup run appends a new timestamped entry.

Q110. What is the Windows RunOnce registry key?

Ans: It is a special registry location (HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce) where you can register a command that runs ONE TIME on the next Windows startup, then automatically deletes itself.

Q111. What is register_reboot_hook used for?

Ans: When a fix requires a restart, this function registers a RunOnce registry entry so that after the reboot, the agent automatically runs the verification check for that session.

Q112. What registry path is used for the RunOnce hook?

Ans: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce\OSDebugAgentResume

Q113. What command is registered in RunOnce after a fix?

Ans: python agent.py resume — this resumes the session after reboot and runs the verification command to check if the fix worked.

Q114. What is python agent.py resume ?

Ans: This command is used after a reboot. It re-checks the system state for that session, runs the verification command, and updates the session status to VERIFIED_POST_REBOOT or FAILED.

Q115. When does the resume command fire automatically?

Ans: After register_reboot_hook is called, the next time Windows starts, the RunOnce entry fires automatically and runs the resume command — no user action needed.

Q116. What is unregister_reboot_hook used for?

Ans: It removes the OSDebugAgentResume RunOnce registry entry. Called at the start of the resume command so the hook only fires once and does not repeat on every subsequent boot.

Q117. Why is it important to unregister the hook after it fires?

Ans: If you forget to unregister, the resume command would run on EVERY boot, showing old session results every time Windows starts — which would be annoying and confusing.

Q118. What is is_reboot_pending() checking?

Ans: It checks specific Windows registry keys that Windows sets when a restart is required. If these keys exist, it means Windows is waiting for a reboot to complete updates or service changes.

Q119. Which Windows registry keys indicate a pending reboot?

Ans: HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\PendingFileRenameOperations and HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUpdate\Auto Update\RebootRequired

Q120. What happens if a reboot is needed but the user declines?

Ans: The agent saves the session and explains that the fix may not be fully active until after a restart. The RunOnce hook is NOT registered (since the user declined), so manual resume will be needed after rebooting.

■ CATEGORY 6: WinRE Cloud Recovery

Q121. What is Windows Recovery Environment (WinRE)?

Ans: WinRE is a special minimal version of Windows that loads when your PC cannot start normally. It has basic tools to repair Windows. You can open a Command Prompt inside WinRE to run commands.

Q122. How do you enter WinRE on a Windows machine?

Ans: Method 1: Force restart 3 times (hold power button to turn off 3 times). Method 2: Hold Shift while clicking Restart in the Start Menu. Method 3: Boot from a Windows USB drive.

Q123. What is the 1-line cloud rescue command?

Ans: `irm https://raw.githubusercontent.com/Ansh00031/SGU-AI-Thon/main/bootstrap.ps1 | iex` — Run this in WinRE PowerShell. It downloads and launches the full agent automatically.

Q124. What is bootstrap.ps1 and what does it do?

Ans: It is a PowerShell script hosted on GitHub. When run, it: detects your offline Windows drive, downloads a portable Python runtime, downloads the agent code, installs dependencies, and launches the agent.

Q125. What is rescue.bat used for?

Ans: It is a simple batch file (.bat) that contains the 1-line cloud rescue command. Useful for users who are more comfortable with Command Prompt (cmd) than PowerShell.

Q126. How does bootstrap.ps1 find Python on a fresh machine?

Ans: It first tries 'python --version' to see if Python is installed. If not found, it downloads the portable Python package (embeddable ZIP) directly from python.org to a temp folder.

Q127. What is Portable Python and how is it downloaded?

Ans: It is a lightweight version of Python (~15MB) that does not require installation — it runs directly from a folder. The bootstrap downloads it from python.org and extracts it to the temp directory.

Q128. From where are the core agent files downloaded?

Ans: From the GitHub raw content URL:
`https://raw.githubusercontent.com/Ansh00031/SGU-AI-Thon/main/[filename]`. Each core file is downloaded individually.

Q129. What files does bootstrap.ps1 download from GitHub?

Ans: agent.py and all core/ module files: config.py, collector.py, executor.py, llm.py, remediation.py, security.py, snapshot.py, system_paths.py, reboot_manager.py, autostart.py, ui.py, blockchain.py

Q130. Does WinRE have internet access by default?

Ans: Usually yes, if Wi-Fi drivers are loaded. For wired ethernet connections it almost always works. For Wi-Fi in WinRE, you may need to connect manually using the WinRE network settings.

Q131. How does bootstrap.ps1 detect the offline Windows drive?

Ans: It looks for a drive that contains the Windows folder (C:\Windows or D:\Windows) but is not the current active boot drive. It uses Get-PSDrive and Test-Path to scan available drives.

Q132. What is Invoke-WebRequest used for in the bootstrap?

Ans: Invoke-WebRequest (shortened as irm = Invoke-RestMethod) is a PowerShell command that downloads content from the internet. The bootstrap uses it to download Python, agent files, and dependencies.

Q133. What dependencies are installed by bootstrap.ps1?

Ans: typer, rich, pydantic, and python-dotenv — all installed via pip. These are the minimum required packages to run the agent's CLI and display.

Q134. What happens if pip is not available in WinRE?

Ans: The bootstrap first tries to use pip with the portable Python. If pip is not bundled, it downloads get-pip.py from pypa.io and runs it to install pip before installing other packages.

Q135. What is the --user fallback in pip installation?

Ans: If pip install fails due to permission issues, the agent retries with 'pip install --user' which installs packages to the user's local folder (no admin rights needed).

Q136. Can the agent fix a PC that won't boot at all?

Ans: Yes — if you can get into WinRE (which most PCs allow). The bootstrap handles downloading everything, and the agent can scan offline event logs and attempt repairs.

Q137. What types of errors can WinRE mode fix?

Ans: Corrupt Windows Update files (via DISM), broken service configurations, incorrect registry settings, and any error that can be fixed with PowerShell commands applied to the offline Windows drive.

Q138. What is DISM and how would it be used offline?

Ans: DISM (Deployment Image Servicing and Management) is a Windows tool that repairs Windows component stores. In WinRE: DISM /Image:C:\ /Cleanup-Image /RestoreHealth repairs the offline Windows installation.

Q139. What is the rescue.bat shortcut used for in WinRE?

Ans: In WinRE Command Prompt, you can run rescue.bat directly. It contains the PowerShell rescue command pre-formatted so users don't have to type the long URL manually.

Q140. What is the network requirement for WinRE recovery to work?

Ans: You need internet access in WinRE (either ethernet or Wi-Fi). The total download is about 15-25MB (Python + agent files + dependencies). A mobile hotspot works fine.

■ CATEGORY 7: Blockchain & Algorand

Q141. What is Algorand and why was it chosen?

Ans: Algorand is a fast, green, and low-cost blockchain. It was chosen because: transactions confirm in ~3.3 seconds, fees are tiny (~0.001 ALGO ≈ ■0.005), it is environmentally friendly, and it has excellent Python SDK support.

Q142. What is Algorand TestNet vs. MainNet?

Ans: TestNet is a practice version of Algorand where you use fake ALGO tokens (free). MainNet is the real blockchain with real ALGO. We use TestNet for the hackathon to demonstrate the concept without real money.

Q143. What is the AlgoKit Lora Explorer?

Ans: It is a web explorer for the Algorand blockchain built by the Algorand Foundation. You can view any transaction, account balance, or note content at: lora.algokit.io/testnet

Q144. What is the URL for AlgoKit Lora TestNet Explorer?

Ans: <https://lora.algokit.io/testnet> — you can view your transaction at: https://lora.algokit.io/testnet/transaction/YOUR_TX_ID

Q145. What is an Algorand wallet and how is one generated?

Ans: A wallet is a pair of keys: public address (like a bank account number) and private key (like a PIN). The agent generates one automatically using `algosdk.account.generate_account()` and saves it to `.backups/algorand_wallet.json`

Q146. Where is the generated wallet stored securely?

Ans: In `.backups/algorand_wallet.json` inside the project folder. The mnemonic phrase (24-word seed phrase) is stored there. You can also save it in the `.env` file as `ALGORAND_MNEMONIC`.

Q147. What is ALGO and how much does a transaction cost?

Ans: ALGO is Algorand's native currency. Each transaction costs 0.001 ALGO (about ■0.005 or \$0.0001). For TestNet, you get free ALGO from the faucet at bank.testnet.algorand.network

Q148. What is a free TestNet ALGO faucet?

Ans: It is a website that gives you free fake ALGO for testing on TestNet. Visit: bank.testnet.algorand.network, paste your wallet address, and receive 10 free ALGO instantly.

Q149. What is the Algorand Faucet URL used in the project?

Ans: <https://bank.testnet.algorand.network> — paste your wallet address here to receive free TestNet ALGO for testing blockchain features.

Q150. What is a SHA-256 cryptographic hash?

Ans: SHA-256 is a mathematical function that converts any data into a unique 64-character fingerprint. The same data always produces the same fingerprint. Even a tiny change produces a completely different fingerprint — making it tamper-proof.

Q151. What exactly is hashed in the audit proof?

Ans: The SHA-256 hash is computed from: the fix script (`fix.ps1`) + the rollback script (`rollback.ps1`) + session metadata (`metadata.json`). All three files are read and hashed together.

Q152. What is an ARC-0002 structured note?

Ans: ARC-0002 is an Algorand standard for writing structured metadata in transaction notes. The agent writes a JSON object containing: app name, version, session ID, error code, status, SHA-256 hash, and timestamp.

Q153. What JSON fields are included in the blockchain audit note?

Ans: standard (arc-0002), app (AutonomousOSDebugAgent), version (1.0.0), session_id, error_code, status, fix_title, sha256 (hash), and timestamp (UTC).

Q154. What is a zero-ALGO self-transaction?

Ans: It is a blockchain transaction that sends 0 ALGO from your wallet back to your own wallet. The purpose is not to transfer money — it is just to write the audit note into the Algorand blockchain permanently.

Q155. Why send a 0 ALGO transaction instead of a real transfer?

Ans: Because we only need to write data (the audit note) to the blockchain — not transfer any money. A 0 ALGO self-transaction is the cheapest way to permanently store data on Algorand.

Q156. What is anchor_session_on_chain function?

Ans: The main function in core/blockchain.py that: computes the SHA-256 hash, builds the ARC-0002 JSON note, creates a 0-ALGO transaction, signs it with your wallet, submits it to Algorand TestNet, and returns the TX ID and Lora URL.

Q157. What is get_or_create_wallet function?

Ans: It checks if you have an existing Algorand wallet (in .env or .backups/algorand_wallet.json). If not, it generates a new one and saves it. Returns: (address, mnemonic, is_newly_created).

Q158. What is get_wallet_balance function?

Ans: It queries the Algorand TestNet Algod API to get the current ALGO balance of your wallet address. Returns the balance in ALGO (not microALGO).

Q159. What is compute_session_hash function?

Ans: It reads the fix.ps1, rollback.ps1, and metadata.json files from a session's .backups/ folder and computes a combined SHA-256 hash of all three files together.

Q160. What is wait_for_confirmation in the blockchain module?

Ans: After submitting a transaction, this function waits (up to 5 block rounds) for the Algorand network to confirm the transaction. Returns the confirmed transaction info including the round number.

Q161. How long does it take to confirm a transaction on Algorand?

Ans: Usually 3-4 seconds. Algorand has instant finality — once confirmed in a block, the transaction is permanent and can never be reversed or altered.

Q162. What is the Algorand block time?

Ans: Approximately 3.3 seconds per block. This means your audit proof is permanently sealed on the blockchain within about 3-4 seconds of submission.

Q163. What is testnet-api.algonode.cloud used for?

Ans: It is a free public Algorand TestNet API node maintained by Algonode. The agent connects to it to submit transactions and query account balances — no API key or account needed.

Q164. What Python library is used for Algorand interaction?

Ans: py-algorand-sdk (also called algosdk). It provides functions for creating wallets, building transactions, signing them, and submitting to the Algorand network.

Q165. What is py-algorand-sdk?

Ans: The official Python SDK for Algorand. Install with: `pip install py-algorand-sdk`. It is auto-installed by the agent if not found.

Q166. What happens if the TestNet wallet has insufficient ALGO?

Ans: The anchor function checks the balance before trying to submit. If less than 0.001 ALGO (1000 microALGO) is available, it shows a message with the faucet URL to get free TestNet ALGO.

Q167. What is the --anchor flag in the diagnose command?

Ans: Adding --anchor to the diagnose command automatically anchors the fix proof to Algorand TestNet after a successful fix. Example: `python agent.py diagnose 0x80070005 --anchor`

Q168. What is python agent.py blockchain status?

Ans: Shows your Algorand TestNet wallet address, current ALGO balance, AlgoKit Lora account explorer link, and the faucet URL to get free TestNet ALGO.

Q169. What is python agent.py blockchain anchor?

Ans: Manually anchors a session's cryptographic proof to Algorand TestNet. If no session ID is given, it anchors the most recent session. Returns a Lora Explorer transaction link.

Q170. Can this be moved to Algorand MainNet for production?

Ans: Yes. Just change the Algod server URL from `testnet-api.algonode.cloud` to `mainnet-api.algonode.cloud` and use real ALGO. Each anchor would cost about 0.001 ALGO (roughly $\blacksquare$ 0.05) — very affordable for enterprises.

CATEGORY 8: Enterprise & Real-World Impact

Q171. What industries benefit most from this tool?

Ans: IT departments in any company, hospitals (critical medical systems), banks (core banking Windows servers), schools, government offices, and individual home users with Windows PCs.

Q172. Why do enterprises need tamper-proof audit logs?

Ans: By law, many industries (banking, healthcare, defense) must prove what changes were made to IT systems and who authorized them. A local log file can be deleted or edited. A blockchain entry cannot.

Q173. What compliance regulations require system change logs?

Ans: ISO 27001 (Information Security), SOC 2 (Service Organization Controls), PCI DSS (Payment Card Industry), HIPAA (Healthcare), and India's IT Act all require evidence of system change management.

Q174. How does this tool help IT support teams?

Ans: Instead of a Tier-1 support technician spending 45 minutes troubleshooting, the agent diagnoses and fixes the issue in 15 seconds. The support team just reviews the AI-generated fix and approves it.

Q175. How much time does the agent save per incident?

Ans: Typically 30-45 minutes per incident becomes 15-30 seconds. For a company with 100 employees getting 10 OS errors per month, this saves 50+ hours of IT support time per month.

Q176. What is the cost of IT downtime for enterprises?

Ans: Industry studies show IT downtime costs enterprises an average of \$5,600 per minute. Reducing even 1 hour of downtime per month saves tens of thousands of dollars.

Q177. How does this tool prevent unauthorized system changes?

Ans: Every fix requires explicit human Y/N approval. Every fix is recorded in .backups/ and optionally on Algorand. Any unauthorized change would show up as a discrepancy in the blockchain audit trail.

Q178. Can this be deployed across a fleet of enterprise PCs?

Ans: Yes. The agent can be deployed via Group Policy or Microsoft Intune to all Windows machines in an organization. Each machine runs its own agent instance with centralized session logs.

Q179. What is the business model for commercializing this?

Ans: SaaS subscription for IT teams (per-seat or per-device pricing), enterprise on-premise deployment, and an API service where IT ticketing systems can call the agent automatically when errors occur.

Q180. How does this compare to existing tools like SolarWinds?

Ans: SolarWinds and similar tools monitor and alert about problems. Our tool actually diagnoses AND fixes the problem autonomously using AI reasoning. Plus our blockchain audit trail is unique in this space.

Q181. What is the competitive advantage of blockchain-backed logs?

Ans: No competitor in the OS repair/IT automation space currently offers tamper-proof on-chain audit trails. This is a unique differentiator for compliance-heavy industries.

Q182. Can this tool be integrated with ticketing systems like Jira?

Ans: Yes. The agent's Python API can be called programmatically. A Jira webhook could trigger python agent.py diagnose [error_code] when a new ticket is created, auto-diagnosing the issue.

Q183. What is the potential scale of this tool globally?

Ans: There are over 1.6 billion Windows PCs worldwide. Even capturing 0.1% of the enterprise IT automation market represents millions of potential users.

Q184. Can this handle Linux servers in future versions?

Ans: Yes. The architecture is designed to support multiple OS adapters. A Linux adapter would use journalctl (logs), systemctl (services), and bash scripts instead of PowerShell.

Q185. What are the current limitations of the project?

Ans: Windows-only support, requires Python installation, no GUI interface yet, AI reasoning requires API key or local model setup, and WinRE recovery depends on internet access being available.

■ CATEGORY 9: Future Scope & Roadmap

Q186. What is the next planned feature after this hackathon?

Ans: Linux support using systemd/journalctl, a web-based dashboard to view session history and blockchain proofs, and automated patch discovery via Windows Package Manager (WinGet).

Q187. Will the tool support Linux and macOS?

Ans: Yes. Linux support via systemd, journalctl, and bash scripts is next. macOS support via launchd and zsh scripts is planned for a later version.

Q188. What is auto patch discovery via WinGet?

Ans: WinGet is Windows' built-in package manager. The agent could automatically discover outdated or broken software packages and suggest/apply updates as part of the fix workflow.

Q189. Can the tool integrate with Microsoft Intune or SCCM?

Ans: Yes. Both Intune (cloud) and SCCM (on-premise) allow running PowerShell scripts on managed devices. The agent can be triggered remotely from these management platforms.

Q190. What is an enterprise SaaS model for this tool?

Ans: A cloud service where enterprises pay per device per month. The service provides: AI diagnostics, session history dashboard, blockchain audit logs, team approval workflows, and compliance reports.

Q191. Can the Algorand anchor move to MainNet for production use?

Ans: Yes. Simply change the API endpoint from testnet to mainnet. Each anchor costs ~0.001 ALGO (~\$0.05). For an enterprise with 1000 repairs/month, total blockchain cost is about \$50/month.

Q192. Will there be a GUI or web dashboard version?

Ans: Yes. A React/Next.js web dashboard is planned that shows: session timeline, active vs solved issues, blockchain transaction history with Lora links, and team member access controls.

Q193. Can the tool predict errors before they happen?

Ans: Proactive monitoring is planned: the agent runs startup-monitor on a schedule, analyzes warning-level events (not just errors), and alerts users before a warning escalates to a critical failure.

Q194. What is the plan for multi-language support?

Ans: The terminal output (Rich panels and tables) will support localization. Hindi, Tamil, Bengali, and other Indian languages are priorities given the Build with Bharat 2.0 focus.

Q195. Could this tool be integrated into a Windows 11 native widget?

Ans: Potentially yes. Windows 11 supports widgets and taskbar system tray apps. A lightweight tray icon that shows system health status and alerts on errors is a planned UX improvement.

CATEGORY 10: Judge Q&A: — Top 5 Critical Questions

Q196. What if the AI gives a harmful command and deletes important files?

Ans: This cannot happen because of 3 safety layers: (1) All diagnostic probes are read-only — they never change anything. (2) Every command is checked against our regex blacklist that blocks del, format, rmdir and similar destructive commands. (3) Even if somehow a risky fix was proposed, you must type Y to approve it — and you can see the full script before approving. Plus, we auto-create a rollback script before any fix runs.

Q197. Why not just use Windows built-in System Restore?

Ans: System Restore has 4 major problems: (1) It doesn't tell you WHY something broke — no AI diagnosis. (2) It restores the ENTIRE system to a past state, which can undo your personal settings and installed software. (3) It has no blockchain audit trail for compliance. (4) It doesn't work in WinRE for boot loop scenarios. Our agent fixes ONLY the broken component, preserves everything else, and provides verifiable proof.

Q198. Is this tool safe to run on a production server?

Ans: Yes, completely safe. Here's why: Read-only diagnostic probes run first with zero system changes. Every proposed fix requires mandatory human Y/N approval. Pre-fix snapshots are created automatically before anything changes. Instant rollback is available if needed. The blockchain audit trail proves exactly what was changed, when, and by whom.

Q199. How is this different from just writing a PowerShell script yourself?

Ans: To write a fix yourself you'd need to: know which of 1000+ Event Viewer log channels to check, manually read and interpret cryptic error messages, know which specific PowerShell commands fix which specific errors, test the fix safely, write a rollback script, and document everything for compliance. Our agent does all of this in 15 seconds, grounded in real live telemetry, with AI reasoning, automatic safety checks, and blockchain-verified documentation.

Q200. Why does an OS debugging tool need blockchain?

Ans: In enterprise IT and regulated industries, 'trust but verify' is not enough — you need 'cryptographic proof'. A local log file can be deleted, edited, or forged. An Algorand blockchain entry is permanent, mathematically tamper-proof, and independently verifiable by anyone with the transaction ID. No IT vendor, no employee, and no hacker can change or delete what is sealed on-chain. This is what makes our tool enterprise-grade.

Team KernelHealers

Autonomous OS Debugging Agent Blockchain & Web3 — Agentic Solutions Track

github.com/Ansh00031/SGU-AI-Thon | lora.algokit.io/testnet